

TrueCompass

Especificação algorítmica e arquitetural do sistema de mobilidade urbana

Grafos e Dijkstra | Árvores (Quadtree) | Ordenação Topológica
Indução Matemática | Análise de Recorrência

Sumário: 1. Introdução — 2. Modelagem em Grafos — 3. Estrutura de Árvore (Quadtree) — 4. Ordenação Topológica — 5.
Fundamentação por Indução Matemática — 6. Análise de Recorrência — 7. Arquitetura do Sistema

1. Introdução

Este documento descreve, em nível técnico, a modelagem algorítmica do TrueCompass, um sistema de mobilidade urbana que conecta passageiros a motoristas. O sistema integra cinco fundamentos de matemática discreta e algoritmos: grafos, árvores, ordenação topológica, indução matemática e recorrência. Cada seção apresenta a definição formal do conceito, sua aplicação específica no sistema, pseudocódigo de referência e a análise de complexidade correspondente.

O sistema é decomposto em três módulos algorítmicos centrais: (i) roteamento sobre grafo ponderado via Dijkstra; (ii) indexação espacial via quadtree para busca de motoristas; e (iii) controle de fluxo de estados da corrida via ordenação topológica sobre um grafo acíclico dirigido (DAG).

2. Modelagem em Grafos e Algoritmo de Dijkstra

2.1 Definição formal

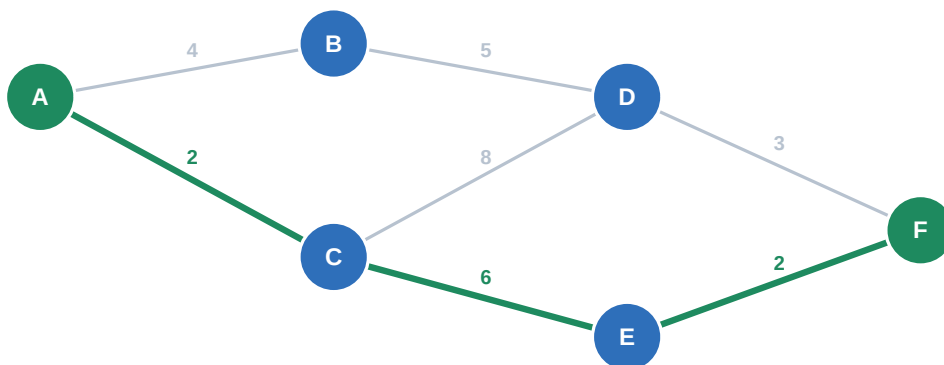
A malha viária é modelada como um grafo ponderado direcionado $G = (V, E, w)$, onde V é o conjunto de vértices (cruzamentos), $E \subseteq V \times V$ é o conjunto de arestas (trechos de rua) e $w: E \rightarrow \mathbb{R}^+$ é uma função peso que associa a cada aresta um custo (distância ou tempo estimado).

A representação adotada é a lista de adjacência, com custo de espaço $O(V + E)$, adequada para grafos esparsos como malhas viárias reais, nas quais cada cruzamento se conecta a poucas ruas.

2.2 Algoritmo de Dijkstra

Dados um vértice de origem s (posição do motorista) e um vértice de destino t (posição do passageiro ou do próximo ponto da corrida), o algoritmo de Dijkstra calcula a menor distância de s a todos os demais vértices alcançáveis, utilizando uma fila de prioridade (min-heap) para selecionar, a cada iteração, o vértice ainda não visitado de menor distância estimada.

```
FUNÇÃO dijkstra(G, s):  
  dist[v] <- infinito, para todo v em V  
  dist[s] <- 0  
  Q <- fila_de_prioridade com todos os vértices, chave = dist  
  
  ENQUANTO Q não vazia:  
    u <- EXTRAIR_MINIMO(Q)  
    PARA CADA vizinho v de u:  
      SE dist[u] + w(u, v) < dist[v]:  
        dist[v] <- dist[u] + w(u, v)  
        anterior[v] <- u  
        DECREASE_KEY(Q, v, dist[v])  
  
  RETORNA dist, anterior
```



A = origem (motorista) F = destino Aresta verde = caminho mínimo (custo total = 10)

Execução do Dijkstra a partir de A: caminho de menor custo A-C-E-F, custo total 10.

2.3 Complexidade

Com a fila de prioridade implementada como min-heap binário, cada operação EXTRAIR_MINIMO custa $O(\log V)$ e cada DECREASE_KEY custa $O(\log V)$. Como cada vértice é extraído uma única vez e cada aresta pode gerar, no máximo, uma atualização, a complexidade total é:

$$O((V + E) \log V)$$

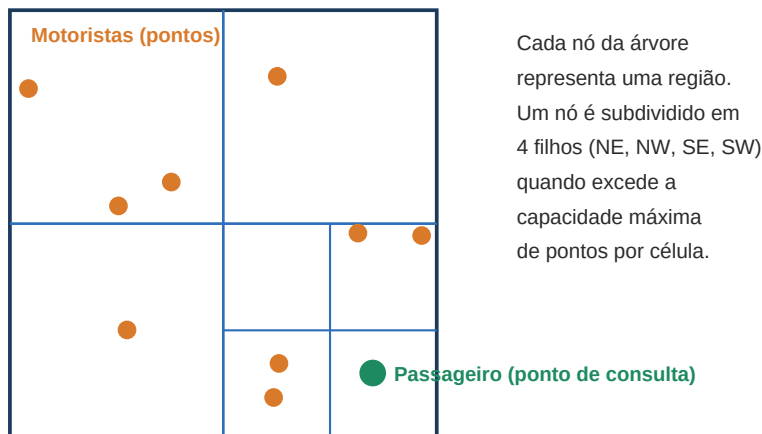
3. Estrutura de Árvore: Quadtree para Busca Espacial

3.1 Definição formal

Uma quadtree é uma árvore em que cada nó interno possui exatamente quatro filhos, correspondentes à subdivisão recursiva de uma região retangular do plano em quatro quadrantes (NE, NW, SE, SW). No TrueCompass, cada nó folha armazena até k motoristas; quando esse limite é excedido, o nó é subdividido em quatro novos quadrantes.

3.2 Operação de busca do vizinho mais próximo

```
FUNÇÃO buscar_motoristas_proximos(nó, ponto_passageiro, raio):  
  SE nó é folha:  
    RETORNA pontos em nó dentro do raio de ponto_passageiro  
  
  resultado <- lista vazia  
  PARA CADA quadrante filho DE nó:  
    SE quadrante.intersecta(ponto_passageiro, raio):  
      resultado <- resultado + buscar_motoristas_proximos(quadrante, ponto_passageiro, raio)  
  
  RETORNA resultado
```



Cada nó da árvore representa uma região. Um nó é subdividido em 4 filhos (NE, NW, SE, SW) quando excede a capacidade máxima de pontos por célula.

Subdivisão recursiva do mapa; a busca descarta ramos cujos quadrantes não intersectam o raio de busca.

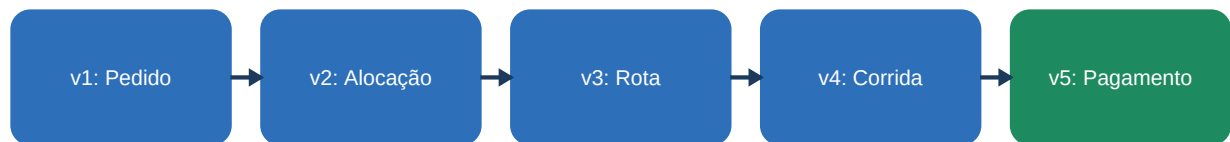
3.3 Inserção e capacidade

Ao inserir um novo motorista, percorre-se a árvore até o nó folha correspondente à sua posição. Se o nó folha já contém k elementos (capacidade máxima), ele é subdividido em quatro filhos e seus pontos são redistribuídos, mantendo o invariante de capacidade em toda a estrutura.

4. Ordenação Topológica do Fluxo da Corrida

4.1 Modelagem como DAG

O ciclo de vida de uma corrida é modelado como um grafo acíclico dirigido $D = (V, E)$, no qual cada vértice representa um estado (pedido, alocação, rota, corrida em andamento, pagamento) e cada aresta (v_i, v_j) indica que o estado v_i é pré-requisito do estado v_j . A aciclicidade de D garante que não existam dependências circulares entre estados.



DAG linear do fluxo da corrida: $v1 \rightarrow v2 \rightarrow v3 \rightarrow v4 \rightarrow v5$.

4.2 Algoritmo de Kahn

A ordenação topológica é obtida pelo algoritmo de Kahn, que processa iterativamente os vértices de grau de entrada zero, removendo-os do grafo e atualizando o grau de entrada de seus sucessores. Se, ao final, existir algum vértice não processado, o grafo contém um ciclo — o que, no domínio da aplicação, indica um estado de corrida inválido.

```
FUNÇÃO ordenacao_topologica_kahn(D):  
    grau_entrada[v] <- número de arestas que chegam em v, para todo v em V  
    fila <- vértices com grau_entrada igual a 0  
    ordem <- lista vazia  
  
    ENQUANTO fila não vazia:  
        u <- REMOVER_PRIMEIRO(fila)  
        ordem.adicionar(u)  
        PARA CADA sucessor v de u:  
            grau_entrada[v] <- grau_entrada[v] - 1  
            SE grau_entrada[v] == 0:  
                fila.adicionar(v)  
  
    SE tamanho(ordem) != tamanho(V):  
        ERRO "ciclo detectado: fluxo de corrida inválido"  
  
    RETORNA ordem
```

Complexidade: $O(V + E)$, pois cada vértice e cada aresta são processados exatamente uma vez.

5. Fundamentação por Indução Matemática

5.1 Corretude do algoritmo de Dijkstra

Proposição: ao término do algoritmo, $\text{dist}[v]$ é igual ao custo do menor caminho de s a v , para todo v alcançável a partir de s .

Prova por indução no número de vértices extraídos da fila de prioridade:

- **Base:** o primeiro vértice extraído é s , com $\text{dist}[s] = 0$, que é trivialmente o menor caminho de s a si mesmo.
- **Hipótese indutiva:** assume-se que, para os k primeiros vértices extraídos da fila, o valor de dist corresponde exatamente ao custo do menor caminho de s até cada um deles.
- **Passo indutivo:** seja u o $(k+1)$ -ésimo vértice extraído, com $\text{dist}[u]$ mínimo entre os vértices restantes na fila. Como todos os pesos são não negativos, qualquer caminho alternativo até u que passe por um vértice ainda não extraído teria custo maior ou igual a $\text{dist}[u]$, pois esse vértice intermediário já teria, por hipótese, distância maior ou igual a $\text{dist}[u]$. Logo, $\text{dist}[u]$ é de fato o menor caminho até u , o que preserva a hipótese para $k+1$ vértices.

Por indução, a propriedade vale para todos os vértices extraídos, incluindo o último, o que conclui a prova de corretude do algoritmo.

5.2 Validade da ordenação topológica

Proposição: se D é um DAG com n vértices, o algoritmo de Kahn sempre produz uma ordenação linear válida, isto é, toda aresta (u, v) satisfaz $\text{posição}(u) < \text{posição}(v)$.

- **Base ($n = 1$):** um único vértice, sem arestas, é trivialmente uma ordenação válida.
- **Hipótese indutiva:** assume-se que o algoritmo produz uma ordenação válida para qualquer DAG com até n vértices.
- **Passo indutivo:** considere um DAG com $n+1$ vértices. Como é um DAG, existe pelo menos um vértice de grau de entrada zero, garantido pela ausência de ciclos. Removendo esse vértice, resta um DAG com n vértices, para o qual, pela hipótese indutiva, o algoritmo produz uma ordenação válida. Reinsere o vértice removido no início dessa ordenação preserva a validade, pois ele não possui predecessores.

Isso garante formalmente que o fluxo pedido $\rightarrow$ alocação $\rightarrow$ rota $\rightarrow$ corrida $\rightarrow$ pagamento sempre pode ser ordenado de forma consistente, para qualquer extensão futura do DAG de estados.

6. Análise de Recorrência da Quadtree

Seja $T(n)$ o tempo necessário para buscar motoristas próximos em uma quadtree com n pontos distribuídos uniformemente. Em cada chamada, a busca desce para um dos quatro quadrantes filhos, processando uma fração aproximadamente igual dos pontos, o que gera a seguinte relação de recorrência:

$$T(n) = T(n / 4) + O(1)$$

Essa recorrência pode ser resolvida pelo Teorema Mestre, na forma $T(n) = a \cdot T(n/b) + f(n)$, com $a = 1$, $b = 4$ e $f(n) = O(1)$. Como $f(n) = O(n^{\log_b(a)}) = O(n^0) = O(1)$, recai-se no caso 2 do Teorema Mestre, resultando em:

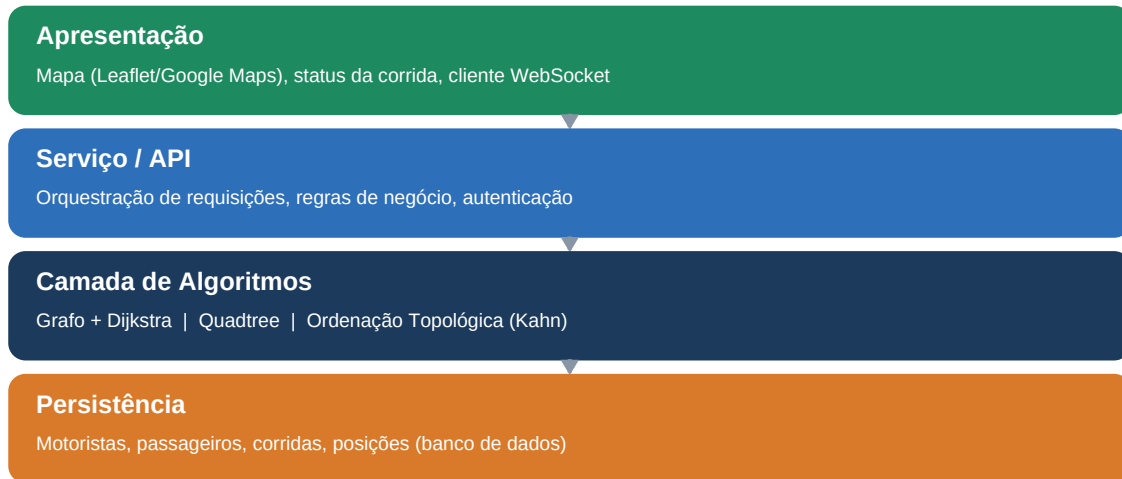
$$T(n) = O(\log n)$$

Isso contrasta com a busca linear ingênua, que compara a posição do passageiro com todos os n motoristas cadastrados, custando $O(n)$. A tabela a seguir resume a comparação:

Estratégia	Complexidade	Observação
Busca linear (todos os motoristas)	$O(n)$	Inviável em escala, com milhares de motoristas simultâneos.
Busca via Quadtree	$O(\log n)$	Descarta ramos inteiros da árvore que não intersectam a região de busca.

7. Arquitetura do Sistema

O sistema é organizado em quatro camadas, seguindo separação de responsabilidades: apresentação, serviço/API, algoritmos e persistência. A camada de algoritmos concentra os três módulos centrais descritos neste documento (grafo/Dijkstra, quadtree, ordenação topológica), isolados da lógica de apresentação e de acesso a dados.



Camadas do sistema: cada uma consome serviços apenas da camada imediatamente inferior.

7.1 Tecnologias de referência

Camada	Tecnologias de referência
Apresentação	Leaflet.js ou Google Maps Platform (mapa); WebSocket (status em tempo real)
Serviço / API	API REST/HTTP em Python (framework de backend a definir pelo squad, ex.: FastAPI ou Flask)
Algoritmos	Implementação própria de Dijkstra, quadtree e ordenação topológica (Kahn) em Python
Persistência	Banco de dados MySQL para motoristas, passageiros, corridas e posições

7.2 Quadro de Responsabilidades da Equipe

Detalhamento da função de cada participante dentro do squad.

Participante(s)	Área de Responsabilidade	Função dentro do Squad
Bianca Murcela Mascamudo e Carolina Yukari Kague	Lógica Matemática — Matemática Computacional Aplicada	Responsáveis pelo desenvolvimento e aplicação da lógica matemática no projeto, utilizando conceitos de Matemática Computacional para estruturar e solucionar os problemas relacionados ao sistema.
Wildiner Lopes e Gabryella Araújo	Front-end	Responsáveis pelo desenvolvimento da interface do sistema, incluindo a estruturação das telas, os elementos visuais e a interação do usuário com a aplicação.
Kauan Camargo e Carolina Chaves	Back-end	Responsáveis pela implementação da parte interna do sistema, incluindo regras de negócio, processamento das informações e integração entre as funcionalidades da aplicação.